Studio Nx

JS 14 — initLoginModalFromHash()

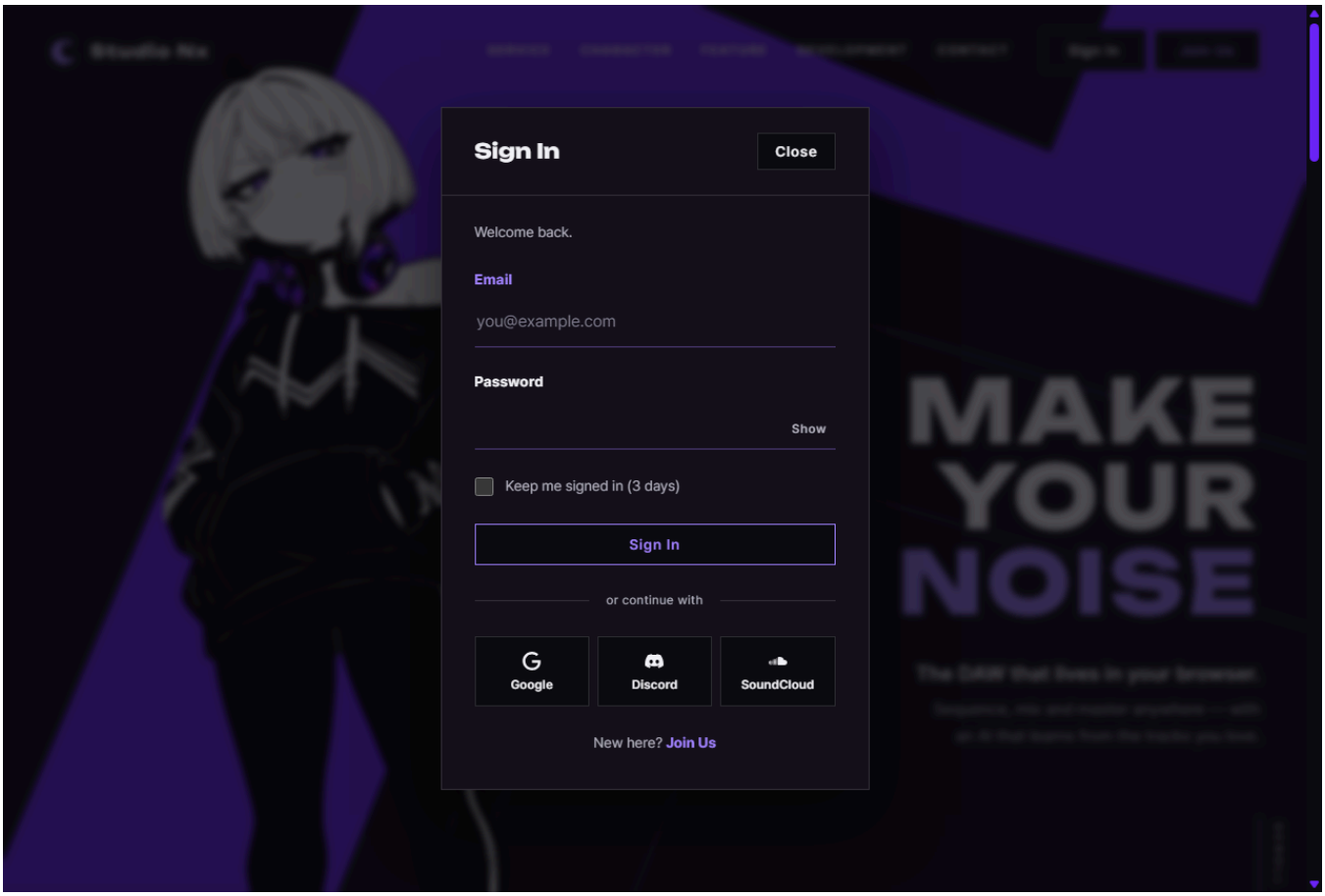
2026-10-12

JS 14 — `initLoginModalFromHash()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-103: 로그인이 필요한 페이지 → 로그인 → 원래 페이지)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	13 `initLoginForm()` · 15 `initDuplicateChecks()`

1. 하는 일

주소가 `home.html#login…`이면 **로그인 모달을 자동으로 엽니다.** 모달을 닫으면 주소에서 `#login…`을 지웁니다.



들어온 주소	결과
`home.html`	모달이 닫혀 있습니다 (평소대로)
`home.html#login`	모달이 열립니다
`home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new`	모달이 열리고, 로그인하면 `/myProject.html?sort=title#new`로 갑니다 (13)
`home.html#features`	모달이 닫혀 있고, 그 구역으로 스크롤만 합니다

2. 언제 쓰나

로그인이 필요한 페이지가 로그인하지 않은 사람을 홈으로 보낼 때 이 주소를 씁니다. 이 함수는 **P1-02, P1-05, P1-07이 이어지는 흐름의 가운데 고리**입니다.

순서	화면	일어나는 일	관련
①	myProject.html (로그인하지 않음)	`requireLogin()` 이 재발급에서 401을 받고 `home.html#login?next=…` 로 이동합니다	JS 07, JS 04
②	home.html	**이 함수**가 모달을 자동으로 엽니다	
③	로그인 모달	로그인에 성공하면 주소의 `next` 로 이동합니다	JS 13, JS 05
④	myProject.html	`requireLogin()` 이 재발급에서 200을 받고 그대로 진행합니다	

로그인하지 않고 **Close**를 누르면 주소가 `home.html` 로 바뀝니다. 그래서 새로 고침해도 모달이 다시 열리지 않습니다.

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash } from '../core/auth.js';
```

`parseLoginHash` 줄을 이번에 추가했습니다. 이 함수는 그것만 씁니다.

함수

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
  if (!modal) return;

  const openIfLoginHash = () => {
    if (parseLoginHash().open && !modal.open) modal.showModal();
  };

  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) {
      history.replaceState(null, '', location.pathname + location.search);
    }
  });
  window.addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

`home.js` 아래쪽에서 이미 `includePartials()`(모달 HTML 끼우기) 다음에 이 함수를 부르고 있습니다. 모달이 있어야 찾을 수 있으므로 순서가 중요합니다.

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
  if (!modal) return;
  const openIfLoginHash = () => parseLoginHash().open && !modal.open && modal.showModal();
  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) history.replaceState(null, '', location.pathname + location.
search);
  });
  addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

최신 문법	실제 코드	뜻
`조건 && 조건 && 실행()`	`if (조건 && 조건) 실행()`	`&&`를 "참이면 다음을 실행"하는 데 쓰는 짧은 습관입니다. 읽기 어려워서 실제 코드는 `if`를 씁니다
`addEventListener(…)` (앞에 `window.` 가 없음)	`window.addEventListener(…)`	전역 함수는 `window.`를 생략할 수 있습니다

이 함수에는 최신 문법만의 기능이 거의 없습니다. 차이는 "짧게 쓰는 습관" 정도입니다.

문법 풀이

import 문법	뜻
`import { parseLoginHash } from '../core/auth.js'`	`../`는 한 폴더 위(`static/js/`)라는 뜻이므로, `static/js/core/auth.js`를 가리킵니다

식	뜻
`const openIfLoginHash = () => { … }`	이름을 붙인 화살표 함수입니다. 아래에서 두 번(처음, `hashchange`) 씁니다
`modal.showModal()`, `modal.open`	`<dialog>`를 모달로 열고, 열려 있는지 확인합니다
`modal.addEventListener('close', …)`	모달이 **어떤 이유로든** 닫힐 때 실행합니다 (Close 버튼, Esc, 로그인 성공 후의 `close()`)
`location.hash.startsWith('#login')`	주소의 `#` 부분이 `#login`으로 시작하는지 확인합니다
`history.replaceState(null, '', 새 주소)`	페이지를 다시 열지 않고 **주소 글자만** 바꿉니다. 방문 기록도 새로 쌓지 않습니다
`window.addEventListener('hashchange', …)`	페이지 안에서 `#…` 부분만 바뀔 때 실행합니다

4. 흐름

1. 모달이 없으면 끝냅니다.
2. 모달이 닫힐 때 주소에서 `#login…`을 지우도록 연결합니다.
3. 주소의 `#` 부분이 바뀔 때 다시 확인하도록 연결합니다.
4. 지금 주소를 확인해서 `#login`이면 모달을 엽니다.

5. 왜 이렇게

선택	이유
`next`를 따로 저장하지 않음 (템플릿에는 "저장해 두었다가"라고 되어 있었음)	모달이 열려 있는 동안 주소에 `#login?next=…`가 그대로 있습니다. 13번이 로그인에 성공한 순간 `parseLoginHash()`로 직접 읽으므로, 저장 장소를 하나 더 둘 필요가 없습니다
닫으면 `#login…`을 지움	그대로 두면 새로 고침할 때마다 모달이 다시 뜹니다
`replaceState`를 씀 (`location.hash = ''`가 아님)	`hash`를 바꾸면 방문 기록이 한 칸 쌓이고, 주소 끝에 `#`가 남습니다. `replaceState`는 둘 다 생기지 않습니다
`hashchange`도 받음 (템플릿에는 없었음)	이미 home.html을 보고 있을 때 `#login` 링크를 누르거나 주소창에 입력해도 모달이 열립니다
`!modal.open`을 확인	이미 열린 모달에 `showModal()`을 또 부르면 브라우저 오류가 납니다
바깥 주소인 `next`는 걸러짐	판단은 JS 05를 거쳐 JS 03 `safeNextPath`가 합니다. 모달은 열리지만 `next`는 `null`이므로, 로그인하면 홈에 머뭅니다

6. 시험 (실제 home.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 8개)

시험	확인한 것
`home.html#login?next=%2FmyPage.html`로 열기	모달이 열리고, `parseLoginHash()`는 `{open: true, next: "/myPage.html"}`입니다
닫기	주소가 `/home.html`이 됩니다 (해시 없음)
페이지 안에서 `#login`으로 이동	모달이 열리고(`hashchange`), 닫으면 다시 지워집니다
`#features`	모달이 열리지 않습니다
`#login?next=https%3A%2F%2Fevil.com`	모달은 열리고, `next`는 `null`입니다
`home.html`(해시 없이)로 열기	모달이 닫혀 있습니다

8개 모두 통과했습니다. 위의 캡처는 `home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new`로 열고, 첫 접속 로딩 화면이 끝난 뒤에 찍었습니다.